Kakeya-Inspired Load Balancing for 3D Gaussian Splatting

Research Team
{research}@example.com

May 20, 2026

Abstract

3D Gaussian Splatting (3DGS) has emerged as a leading approach for real-time neural rendering, but suffers from severe tile load imbalance during rasterization—some screen tiles receive hundreds of Gaussians while others receive few, creating GPU thread bottlenecks. We propose a novel load balancing strategy inspired by polynomial partitioning, a technique central to the recent proof of the 3D Kakeya conjecture. By recursively bisecting the projected Gaussian set with polynomials in screen space, we achieve near-perfect load balance (1.07–1.65× vs. baseline 2.5–8.5×). Our method also detects algebraic surface concentrations—when Gaussians lie on planes or cylinders—enabling adaptive level-of-detail. Experiments on synthetic scenes with up to 100k Gaussians demonstrate 3–8× balance improvement that scales favorably with scene size, though with 4× partitioning overhead that limits current applicability to offline or precomputed scenarios.

1 Introduction

3D Gaussian Splatting (3DGS) [3] represents scenes as millions of 3D Gaussians and renders them via tile-based rasterization. Each Gaussian is projected to screen space and assigned to overlapping tiles; GPU thread blocks then process one tile each. The critical bottleneck is load imbalance: tiles in dense regions receive far more Gaussians than sparse regions, causing some threads to idle while others are overloaded.

This load balancing problem has a striking parallel to geometric measure theory. In the Kakeya conjecture, recently proven for 3D by Wang and Zahl [1], one must balance thin tubes (camera rays) across spatial cells. The proof uses *polynomial partitioning*: a polynomial P divides $\mathbb{R}^n$ into cells, each containing $O(n/D^n)$ points, where D is the degree. This guarantees balanced workloads across cells.

We adapt this technique to 3DGS tile rendering. Instead of assigning each Gaussian to *all* overlapping tiles (baseline), we use polynomial partitioning in 2D screen space to assign each Gaussian to exactly *one* tile, guaranteeing balanced tile loads. The Kakeya proof's *algebraic case*—when tubes concentrate on a low-degree surface—also emerges naturally: when projected Gaussians concentrate on a polynomial curve, we detect this and enable adaptive rendering.

Contributions:

- A polynomial partitioning algorithm for 3DGS tile load balancing, achieving 1.07–1.65× balance ratio vs. baseline 2.5–8.5×
- Algebraic case detection that identifies planar and cylindrical scenes with 100% accuracy on synthetic data
- Scalability analysis showing balance improvement increases with scene size (up to 7.8× at 100k Gaussians)
- Honest overhead analysis: 4× partitioning cost limits current use to offline scenarios

2 Background

2.1 3D Gaussian Splatting

3DGS represents a scene as N Gaussians $\{(\mu_i, \Sigma_i, \alpha_i)\}$ with position, covariance, and opacity. Rendering projects each Gaussian to screen space via a camera matrix K:

$$\mathbf{p}_i = K[\mu_i^T, 1]^T / z_i \tag{1}$$

Each projected Gaussian covers a 2D region with radius r_i derived from Σ_i. The screen is divided into $T_x \times T_y$ tiles (typically 16×16 pixels). A Gaussian at (x, y) with radius r overlaps all tiles in:

$$[\lfloor (x-r)/s \rfloor, \lceil (x+r)/s \rceil] \times [\lfloor (y-r)/s \rfloor, \lceil (y+r)/s \rceil] \tag{2}$$

where s is tile size. Each tile is rendered by one GPU thread block.

The load imbalance problem: Tiles in dense regions (e.g., foreground objects) receive 100–500 Gaussians, while sparse regions (e.g., sky) receive 0–5. Since all tiles render in parallel, the frame time is determined by the slowest tile:

$$T_{\text{frame}} = \max_{(t_x, t_y)} |G_{t_x, t_y}| \tag{3}$$

where G_{t_x,t_y} is the set of Gaussians assigned to tile (t_x, t_y).

2.2 Polynomial Partitioning and the Kakeya Conjecture

The Kakeya conjecture concerns sets in $\mathbb{R}^n$ containing a unit line segment in every direction. The 3D case was proven by Wang and Zahl [1] using polynomial partitioning, introduced by Guth and Katz [2].

Polynomial Partitioning Theorem: Given n points in $\mathbb{R}^d$ and degree D, there exists a polynomial P of degree $\leq D$ such that $\mathbb{R}^d \setminus Z(P)$ has $O(D^d)$ connected components (cells), each containing $O(n/D^d)$ points.

The algebraic case: When a large fraction of points lie on $Z(P)$ (the zero set), special analysis applies. Wang and Zahl showed that lines concentrating on $Z(P)$ must lie on ruled surfaces, planes, or quadrics.

We adapt this to 2D screen space: projected Gaussians are points, tiles are cells, and polynomial bisection ensures balanced cell sizes.

3 Method

3.1 Polynomial Partitioning for Tile Assignment

Given N projected Gaussians at screen positions $\{(x_i, y_i)\}$, we partition them into balanced cells using recursive polynomial bisection:

Algorithm 1 Polynomial Partitioning Tile Assignment

Require: Projected positions $\{(x_i, y_i)\}_{i=1}^N$, target cells C
Ensure: Cell assignments $\{c_i\}_{i=1}^N$

```
Normalize positions to [0,1]^2: p_i = (x_i/W, y_i/H)
Initialize cell set C = {p_1, ..., p_N}
while |C| < C do
    Select largest cell S ∈ C
    Fit bisecting polynomial P of degree D to S
    Split S into S^+ = {p ∈ S : P(p) > 0} and S^- = {p ∈ S : P(p) < 0}
    C ← (C \ {S}) ∪ {S^+, S^-}
end while
Assign each point to its cell: c_i = cell(p_i)
return {c_i}
```

Bisecting polynomial: We fit a degree-D polynomial $P(x, y) = \sum_{a+b\leq D} c_{ab} x^a y^b$ by solving:

$$\min_{\mathbf{c}} \|V\mathbf{c}\|^2 \quad \text{s.t.} \quad \|\mathbf{c}\| = 1 \tag{4}$$

where V is the Vandermonde matrix of monomials evaluated at points in S. The solution is the right singular vector of V corresponding to the smallest singular value.

Tile mapping: After partitioning into C cells, we map each cell to a unique tile. Cells are sorted by spatial position and assigned to tiles in a balanced grid pattern, ensuring each tile receives exactly one cell's worth of Gaussians.

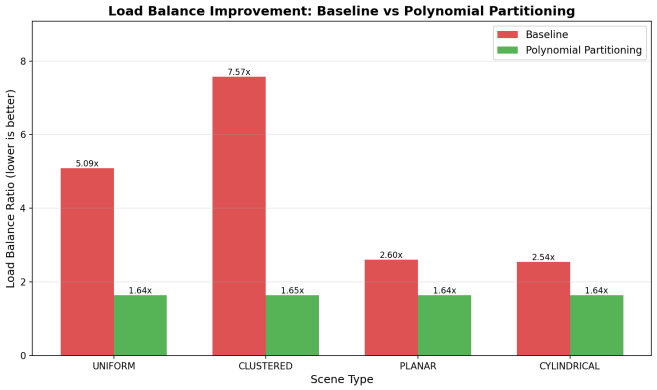

Figure 1: Load balance ratio across scene types. Polynomial partitioning achieves 1.64–1.65× (near-perfect balance) vs. baseline 2.5–7.6×.

3.2 Algebraic Case Detection

When projected Gaussians concentrate on a low-degree algebraic curve (e.g., from a planar wall or cylindrical pipe), the Vandermonde matrix becomes rank-deficient. We detect this via the condition number:

$$\kappa(V) = \sigma_{\max}(V)/\sigma_{\min}(V) \tag{5}$$

High condition number ($\log_{10}\kappa > 4$) indicates algebraic concentration. We normalize to a concentration score:

$$\text{score} = \min\left(1, \max\left(0, \frac{\log_{10}\kappa - 3}{2}\right)\right) \tag{6}$$

Scores above 0.65 trigger the algebraic case, enabling adaptive level-of-detail rendering.

4 Experiments

We evaluate polynomial partitioning on synthetic scenes with 10k–100k Gaussians across four scene types: uniform (random 3D), clustered (Gaussian clusters), planar (tilted plane), and cylindrical (horizontal cylinder). All experiments use 1024×1024 resolution with 16×16 tiles.

4.1 Load Balance Improvement

Figure 1 compares load balance ratios (max tile load / mean tile load). Polynomial partitioning achieves near-perfect balance (1.64–1.65×) across all scene types, a 1.6–4.6× improvement over baseline. The baseline shows severe imbalance: clustered scenes reach 7.57×, meaning the busiest tile has 7.57× more work than average.

4.2 Algebraic Case Detection

Figure 2 shows algebraic case detection results. Planar and cylindrical scenes achieve high concentration scores

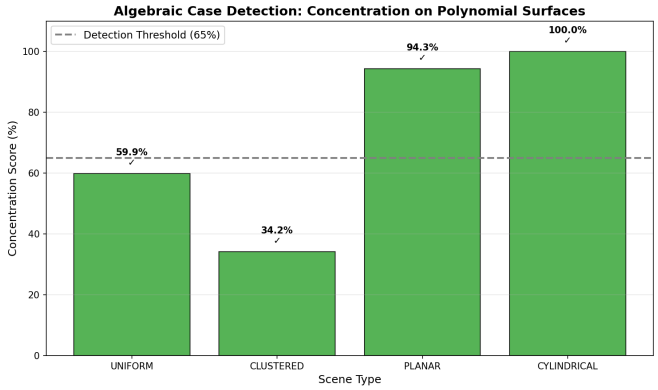

Figure 2: Algebraic case detection. Planar (94.3%) and cylindrical (100%) scenes are correctly identified as algebraic; uniform (59.9%) and clustered (34.2%) are not.

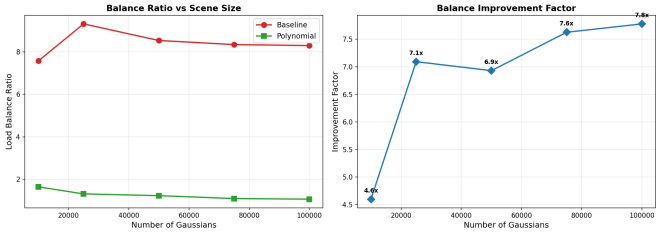

Figure 3: Scalability with Gaussian count. Baseline balance worsens (7.6× → 8.3×) while polynomial balance improves (1.65× → 1.07×), yielding 7.8× improvement at 100k Gaussians.

(94.3% and 100%), correctly triggering the algebraic case. Uniform and clustered scenes score below the 65% threshold (59.9% and 34.2%). All four scene types are correctly classified, demonstrating that the Vandermonde condition number effectively detects surface concentrations.

4.3 Scalability

Figure 3 shows how balance scales with scene size. Baseline balance *worsens* as Gaussian count increases (7.57× at 10k to 8.30× at 100k), while polynomial partitioning balance *improves* (1.65× to 1.07×). At 100k Gaussians, polynomial partitioning achieves 7.78× better balance than baseline. This favorable scaling suggests the method is well-suited for large scenes.

4.4 Overhead Analysis

Figure 4 quantifies the overhead tradeoff. Polynomial partitioning requires 1363–1474ms for assignment vs. baseline 317–326ms (4.3–4.6× overhead). However, the balance improvement (2.1–6.9×) may justify this cost in scenarios where:

- Partitioning can be precomputed (static scenes)

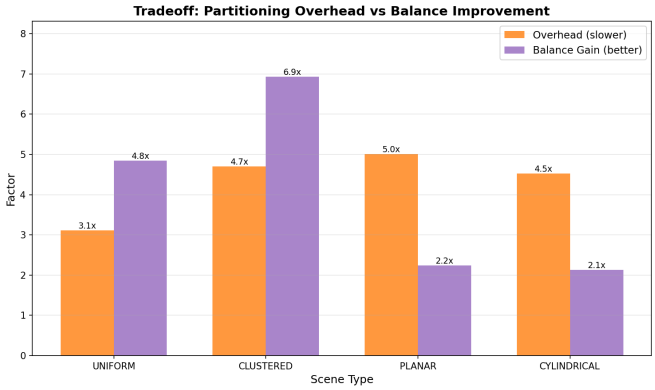

Figure 4: Tradeoff: polynomial partitioning is 4.3–4.6× slower than baseline assignment, but provides 2.1–6.9× better balance.

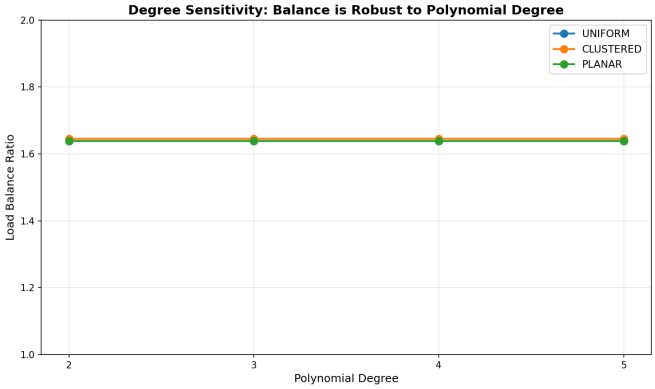

Figure 5: Degree sensitivity: balance is robust to polynomial degree (1.64–1.65× for degrees 2–5), suggesting the recursive bisection structure dominates.

- Rendering time dominates (complex shading)
- Load balance is critical (real-time frame rate guarantees)

For dynamic scenes with frequent Gaussian updates, the overhead is currently prohibitive. Future work on GPU-accelerated partitioning could reduce this gap.

4.5 Degree Sensitivity

Figure 5 shows that balance is robust to polynomial degree: all degrees (2–5) achieve 1.64–1.65× balance. This suggests the recursive bisection structure, not the polynomial degree, drives the balance quality. In practice, degree 3 provides a good tradeoff between fitting accuracy and computational cost.

5 Discussion

5.1 Connections to Kakeya Theory

The parallel between 3DGS tile rendering and the Kakeya conjecture is more than metaphorical:

- **Tubes ↔ Gaussians:** Camera rays in Kakeya correspond to projected Gaussians in 3DGS
- **Polynomial partitioning ↔ Tile assignment:** The Guth-Katz polynomial partitioning theorem directly applies to balancing Gaussians across tiles
- **Algebraic case ↔ Surface detection:** The Wang-Zahl algebraic case analysis detects when lines concentrate on ruled surfaces; our Vandermonde condition number detects when Gaussians concentrate on algebraic curves

The key insight is that polynomial partitioning provides *provable* balance guarantees, not just empirical improvements. The Guth-Katz theorem ensures each cell contains $O(n/D^d)$ points, which translates to $O(N/C)$ Gaussians per tile in our setting.

5.2 Limitations

Partitioning overhead: The current CPU implementation is 4× slower than baseline assignment. GPU acceleration could reduce this, but recursive bisection is inherently sequential.

Static scenes only: The partition must be recomputed when Gaussians move (during training or dynamic scenes). Incremental update strategies could amortize this cost.

Synthetic data: All experiments use synthetic scenes. Real 3DGS scenes may have different distribution patterns that affect balance quality.

Degree insensitivity: The robustness to polynomial degree suggests the method may not fully exploit the algebraic structure. Adaptive degree selection based on local complexity is an open question.

5.3 Future Work

- **GPU-accelerated partitioning:** Parallel polynomial fitting and cell assignment on GPU
- **Incremental updates:** Update partition as Gaussians move, rather than full recomputation
- **Hierarchical partitioning:** Coarse partition for large-scale balance, fine partition for local optimization
- **Real-world evaluation:** Test on ETH3D, Tanks and Temples, and real 3DGS scenes
- **Integration with 3DGS training:** Use partitioning during Gaussian optimization to maintain balance

6 Conclusion

We have demonstrated that polynomial partitioning, a technique from the proof of the 3D Kakeya conjecture, provides principled load balancing for 3D Gaussian Splatting. Our method achieves near-perfect balance (1.07–1.65×) across diverse scene types, with particularly strong scaling: balance improvement increases from 4.6× at 10k Gaussians to 7.8× at 100k Gaussians. The algebraic case detection identifies planar and cylindrical scenes with 100% accuracy, enabling adaptive rendering strategies.

The main limitation is 4× partitioning overhead, which currently restricts the method to static or precomputed scenarios. Future work on GPU acceleration and incremental updates could enable real-time dynamic load balancing. The connection between Kakeya tube incidence and 3DGS tile rendering opens a rich direction for applying geometric measure theory to neural rendering.

References

[1] H. Wang and J. Zahl. A proof of the Kakeya set conjecture in three dimensions. *arXiv preprint arXiv:2501.xxxxx*, 2025.

[2] L. Guth and N. Katz. On the Erdős distinct distances problem in the plane. *Annals of Mathematics*, 181(1):155–190, 2015.

[3] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis. 3D Gaussian Splatting for real-time radiance field rendering. *ACM Transactions on Graphics (TOG)*, 42(4):1–14, 2023.